

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом
Вариант 1и

Студент гр. 4345

Кравцов С. А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

Цель работы

Изучить и реализовать алгоритм поиска с возвратом. Оптимизировать.

Постановка задачи

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

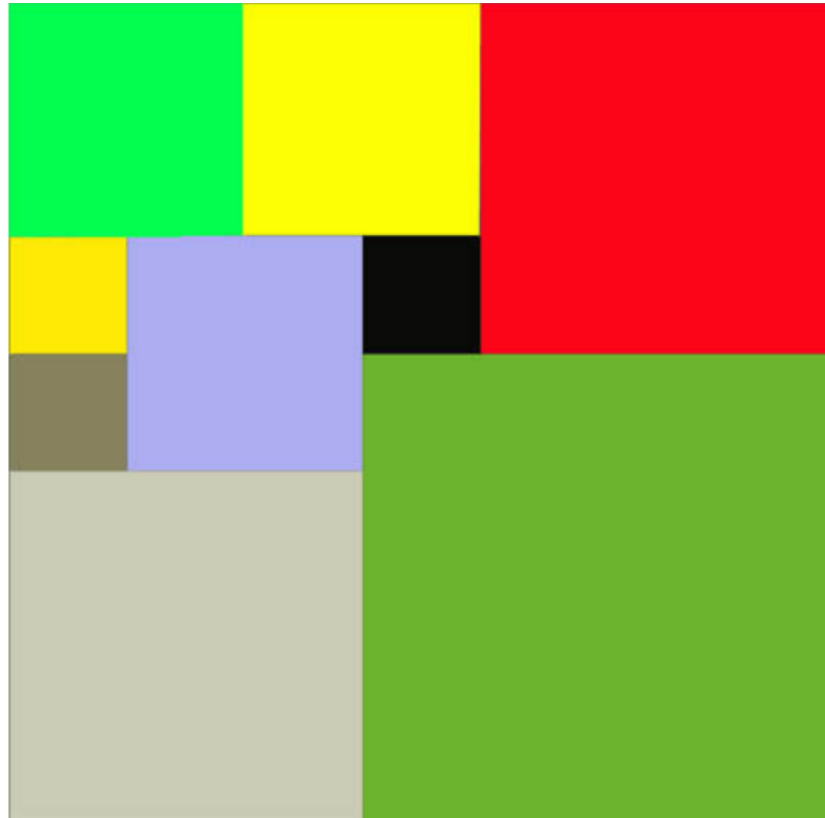


Рисунок 1 – Столешница 7×7

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых

числа x, y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 1и - Итеративный бэктрекинг. Выполнение на Stepik двух заданий в разделе 2.

Выполнение работы

Для оптимизации задачи заполнения квадрата со стороной N квадратами со сторонами в диапазоне от 1 до $N - 1$ с помощью поиска с возвратом, можно ее разбить на три случая:

1. N – четное, в таком случае минимально возможное количество квадратов для заполнения столешницы – 4, каждый из которых размера $N/2$. (см Рис 2)

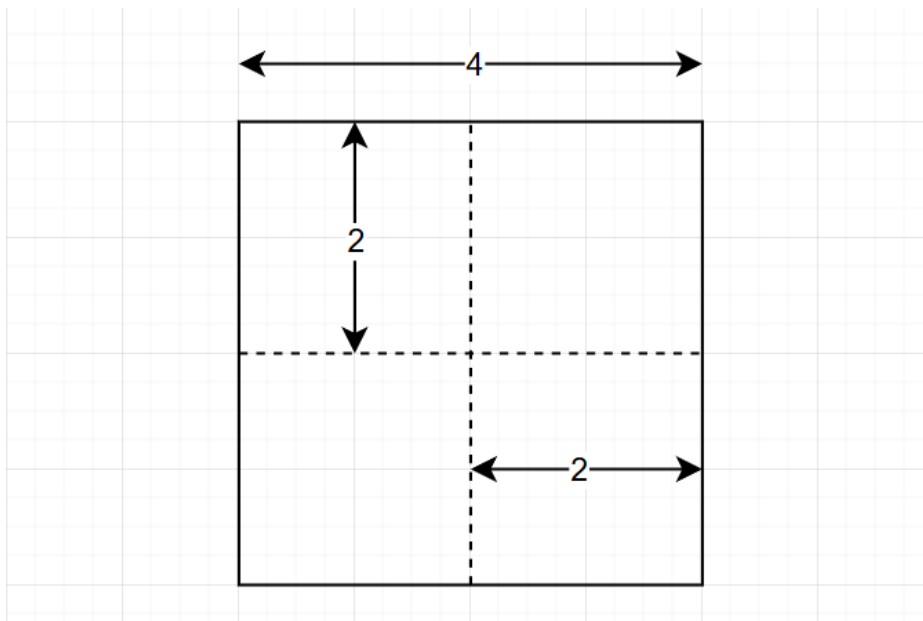


Рисунок 2 – Пример разбиения для $N = 4$

2. N – нечетное, составное, в этом случае нам нужно найти решение для наименьшего простого делителя N и масштабировать (см. Рис 3)

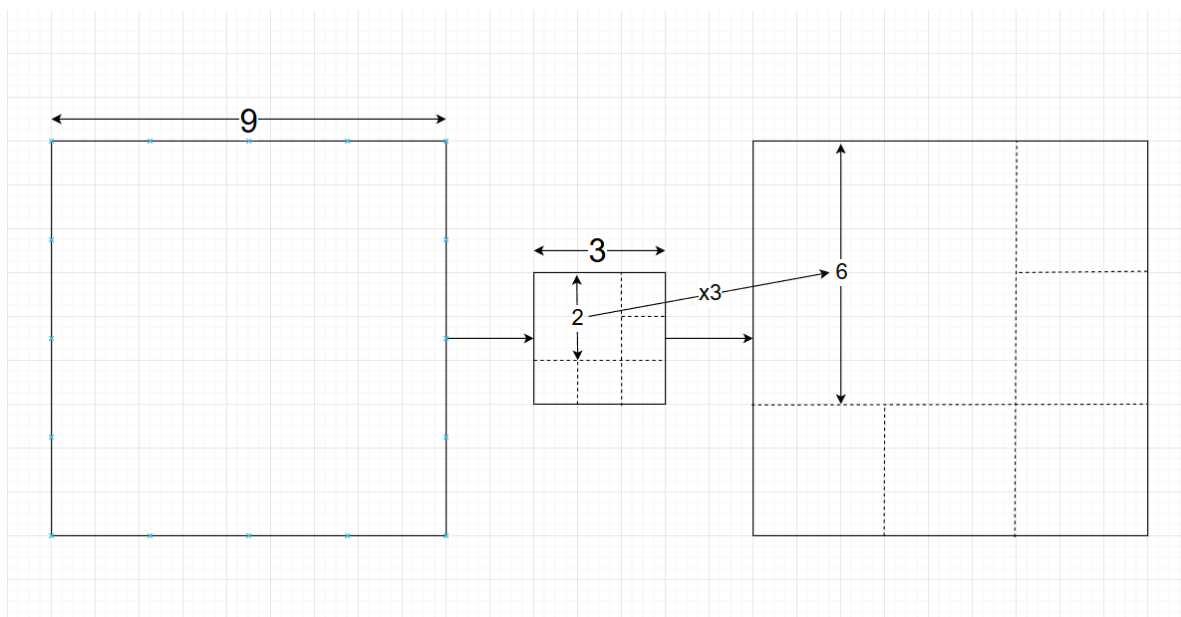


Рисунок 3 – Пример масштабирования для $N = 9$

3. N – нечетное, простое, в этом случае для оптимизации, ставятся 3 квадрата со сторонами $(N+1)/2$, $(N-1)/2$, $(N-1)/2$ в левый верхний, правый верхний и левый нижний угол соответственно, остается лишь небольшая область справа снизу – квадрат со стороной $(N+1)/2$ с исключенным квадратиком со стороной 1 в левом верхнем углу (см Рис. 4):

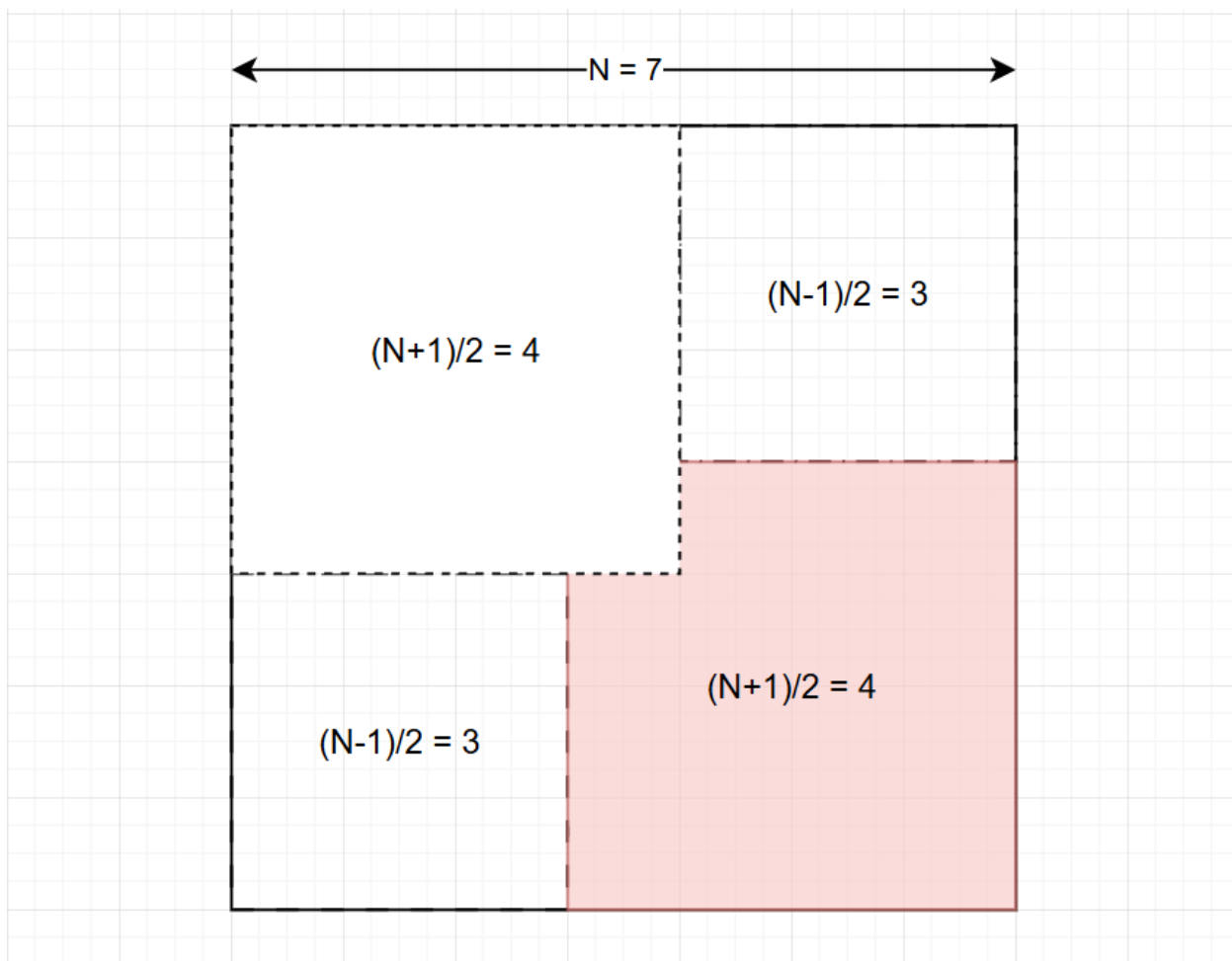


Рисунок 4 – Пример первоначального разбиения для $N = 7$

После начального размещения трёх базовых фрагментов в углах, оставшееся пространство доски обрабатывается методом итеративного перебора с возвратом. Алгоритм находит первую незаполненную ячейку и определяет максимально допустимый размер стороны квадрата, который можно в ней разместить. Далее запускается итерационный процесс: программа поочередно пробует вставить в эту позицию все валидные размеры квадратов — от наибольшего к наименьшему. На каждом шаге применяется оптимизация методом отсечения ветвей: если текущее число фрагментов достигает или превышает уже зафиксированный минимум, ветка отбрасывается. При

достижении полного покрытия доски результат сохраняется как оптимальный, если он превосходит по количеству деталей предыдущее решение.

Для реализации данного алгоритма были созданы:

Функция `place(r: int, c: int, w: int, mode: str = "")` для изменения состояния доски путем размещения или удаления квадрата со стороной `w` и верхним левым углом в координатах `(r, c)`. Операция выполняется с помощью побитового исключающего ИЛИ (XOR) строки `board[i]` с маской, соответствующей ширине квадрата. Если передан параметр `mode`, метод также выводит сообщение о произведенном действии в консоль.

Функция `find_empty()` для поиска первой свободной клетки на столе. Просматривает каждую строку `board`, сравнивая её с маской полностью заполненной строки. При обнаружении незаполненной строки метод вычисляет индекс первого нулевого бита (столбец `c`) и возвращает координаты `(r, c)`. Если все клетки заняты, возвращает `(-1, -1)`

Функция `max_square_w(r: int, c: int)` для определения максимально допустимой стороны квадрата, который можно вписать, начиная с позиции `(r, c)`. Значение ограничивается как границами стола, так и уже занятыми клетками. Проверка осуществляется путем побитового сравнения маски предполагаемой ширины с текущим состоянием строк в диапазоне от `r` до `r + w`.

Сам алгоритм поиска с возвратом, работа которого, описана выше.

Сложность алгоритма по времени $O(c^{(N/4)})$, $c > 1$ – экспоненциальная

Сложность алгоритма по памяти: $O(1)$ для чётных N , $O(N^2)$ для нечётных N

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

№	Входные данные	Выходные данные	Комментарий
1	10	4 1 1 5 1 6 5	Четное число

		6 1 5 6 6 5	
2	$1369 = 37 \cdot 37$	15 (1, 1) 703 (1, 704) 666 (704, 1) 666 (667, 704) 74 (667, 778) 185 (667, 963) 407 (704, 667) 37 (741, 667) 111 (852, 667) 296 (1074, 963) 111 (1074, 1074) 296 (1148, 667) 222 (1148, 889) 37 (1148, 926) 37 (1185, 889) 185	Нечетное составное число, без оптимизации считалось бы вечно) (10^{100} лет примерно)
3	7	9 (1, 1) 4 (1, 5) 3 (5, 1) 3 (4, 5) 1 (4, 6) 2 (5, 4) 1 (5, 5) 1 (6, 4) 2 (6, 6) 2	Нечетное простое число

Выводы

В ходе работы была реализована программа для разбиения квадратной доски на наименьшее количество меньших квадратов с использованием метода поиска с возвратом (backtracking). Алгоритм перебирает все возможные позиции и размеры квадратов, при этом применяя стратегию установки крупных квадратов в начале для сокращения области поиска.

Анализ сложности показал, что в худшем случае алгоритм имеет экспоненциальную сложность, так как количество комбинаций растёт очень быстро с увеличением размера доски. Однако за счёт выбора больших квадратов в начале и обрезки ветвей (когда текущее решение хуже найденного) реальная сложность снижается примерно до $O(c^{N/4})$, где c — среднее число вариантов для каждой позиции.

Таким образом, backtracking оказался эффективным методом для решения задачи разбиения квадрата на меньшие квадраты, позволяя находить оптимальные решения для досок среднего размера.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
import time

N = int(input())
n = N
board = []

def place(r: int, c: int, w: int, mode: str = ""):

    prefix = "Ставим" if mode == "add" else "Убираем"
    if mode: print(f"{prefix} квадрат {w} в ({r+1}, {c+1})")

    mask = ((1 << w) - 1) << c
    for i in range(r, r + w):
        board[i] ^= mask

def find_empty():
    full = (1 << n) - 1
    for r in range(n):
        if board[r] != full:
            row = board[r]
            c = 0
            while (row >> c) & 1:
                c += 1
            return r, c
    return -1, -1

def max_square_w(r: int, c: int):
    max_w = min(n - r, n - c)
    for w in range(1, max_w + 1):
        mask = ((1 << w) - 1) << c
        for i in range(r, r + w):
            if board[i] & mask:
                return w - 1
    return max_w if max_w < N else N - 1
```

```

start = time.time()
multiplier = 1
delimiter = 2

while delimiter * delimiter <= n:
    if n % delimiter == 0:
        multiplier = n // delimiter
        n = delimiter
        break
    delimiter += 1

if n % 2 == 0:
    half = N // 2
    print(f"Четный случай")
    print(f"Квадрат {N}x{N} делится на 4 квадрата со сторонами {half}")
    print(4)
    print(1, 1, half)
    print(1, half + 1, half)
    print(half + 1, 1, half)
    print(half + 1, half + 1, half)
    exit()

print(f"Начало поиска для N={n}, Масштаб x{multiplier}")

board = [0] * n

w1 = (n + 1) // 2
w2 = (n - 1) // 2

place(0, 0, w1)
place(0, w1, w2)
place(w1, 0, w2)

current_res = [(1, 1, w1), (1, w1 + 1, w2), (w1 + 1, 1, w2)]
stack = []
min_k = n * n
best_res = []

```

```

r, c = find_empty()
mw = max_square_w(r, c)
print(f"Первая пустая клетка найдена в ({r+1}, {c+1}). Макс. размер там: {mw}")
place(r, c, mw, mode="add")
stack.append([r, c, mw])
current_res.append((r + 1, c + 1, mw))

while stack:
    k = len(current_res)
    r_empty, c_empty = find_empty()

    if r_empty == -1 and c_empty == -1:
        print(f"Найдено решение: {k} квадратов")
        if k <= min_k:
            min_k = k
            best_res = list(current_res)
            print(f"Новое лучшее решение - {min_k}")

    if k >= min_k:
        while stack:
            print(f'Состояний стека - ', len(stack))
            prev_r, prev_c, curr_w = stack.pop()

            place(prev_r, prev_c, curr_w, mode="del")
            current_res.pop()

            if curr_w > 1:
                next_w = curr_w - 1
                print(f"Уменьшаем квадрат, теперь {next_w} в ({prev_r+1}, {prev_c+1})")
                place(prev_r, prev_c, next_w, mode="add")
                current_res.append((prev_r + 1, prev_c + 1, next_w))
                stack.append([prev_r, prev_c, next_w])
                break
            else:

```

```

        print(f"Все варианты для ({prev_r+1}, {prev_c+1})
исчерпаны, идем выше по стеку")
    else:
        break
else:
    mw = max_square_w(r_empty, c_empty)
    place(r_empty, c_empty, mw, mode="add")
    current_res.append((r_empty + 1, c_empty + 1, mw))
    stack.append([r_empty, c_empty, mw])

print()
print(time.time() - start, "секунд")
print(min_k)
for r, c, w in best_res:
    print(f"(({r - 1} * multiplier + 1), {(c - 1) * multiplier + 1}) {w}
* multiplier}")

```